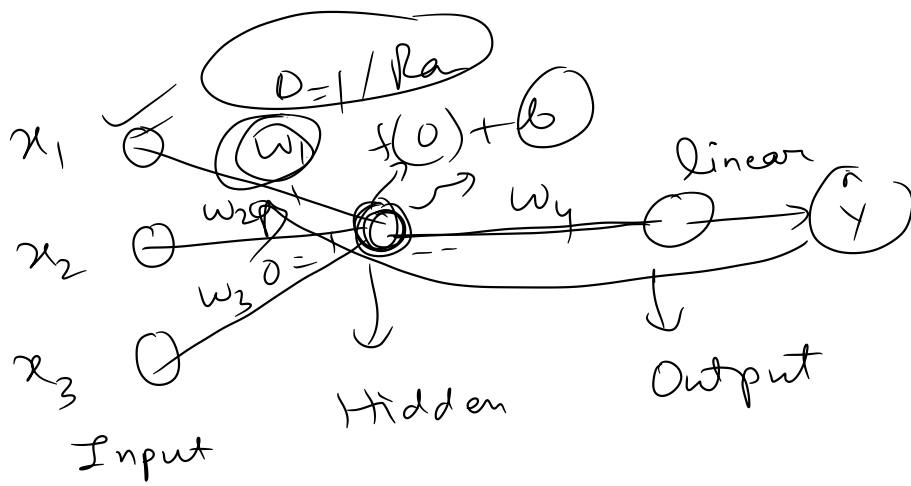




$$\frac{\partial L}{\partial x} = 0$$

$$0 = (\underline{w_1} x_1 + \underline{w_2} x_2 + \underline{w_3} x_3) + b$$

$$\hat{y} = \underline{w_y} \times f(0)$$

$$L = \frac{(y - \hat{y})^2}{2}$$

$$\downarrow \hat{w}_n = w_n - \eta \frac{\partial L}{\partial w_n}$$

$$w_n = w_n - \eta (+ve)$$

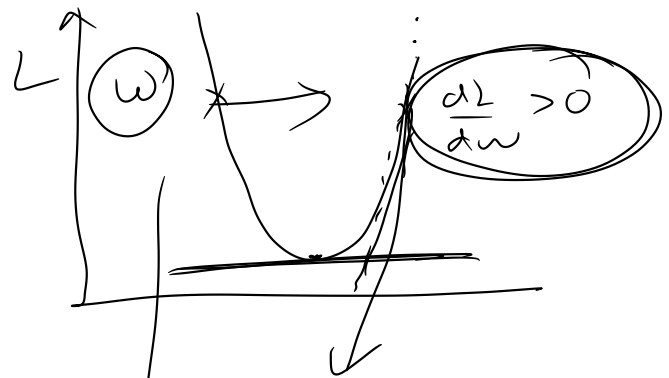
$$w_n = w_n - \eta (-ve) = w_n + \eta (+ve)$$

$$w_y = w_y - \eta \frac{\partial L}{\partial w_y}$$

$$\frac{\partial L}{\partial w_y} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_y}$$

$$\frac{\partial L}{\partial w_n}$$

$$w_y = \underline{\underline{w_y}}$$



$$\frac{dL}{dw} < 0$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{-2(y - \hat{y})}{2} = -1(y - \hat{y})$$

$$\underline{\underline{dL}} = -A$$

$$\hat{y} = w_y f(0)$$

$$\frac{\partial \hat{y}}{\partial w_y} = w_y f'(0)$$

$$\frac{d\hat{y}}{dw_y} = f(0)$$

$$w_4 = \neq$$

$$w_1, w_2, w_3$$

$$L = \frac{(\hat{y} - y)^2}{2}$$

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$\frac{\partial \hat{y}}{\partial w_1} = \frac{\partial}{\partial w_1} \left(w_1 f(\phi) \right)$$

$$\frac{\partial \hat{y}}{\partial \phi} = w_4 f'(\phi)$$

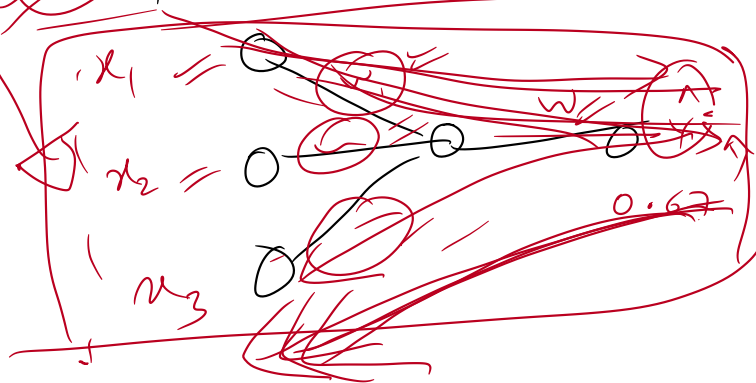
$$f'(x) = \frac{1}{1 + e^{-x}}$$

$$\hat{y} = w_4 f(\phi)$$

$$\phi = w_1 x_1 + w_2 x_2 + w_3 x_3$$

$$\frac{\partial \phi}{\partial w_1} = x_1$$

	Gender	Ticket Price	Age	Survived
A	M	£500	40	0
B				0
C				0

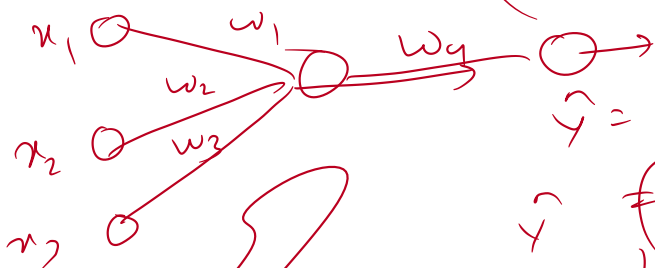


$$L = -y \log \hat{y} - (1-y) \log (1-\hat{y})$$

$$(1) L = -1 \log 0.67 - (1-1) \log (1-0.67)$$

$$L = -\log 0.67$$

$$\phi = (w_1 x_1 + w_2 x_2 + w_3 x_3)$$



$$\hat{y} = w_4 f(\phi)$$

$$\hat{y} = w_4 (w_1 x_1 + w_2 x_2 + w_3 x_3)$$

Epochs

1 Epoch

Batch

Gradient Descent

Stochastic (SGD)

Epoch = 5

Lang

1 row

Mini batch

400

100

4 batches

100 $\rightarrow$ y_n
 $\leftarrow w$

4 weights

5 epochs $\rightarrow$ 20 weights

400

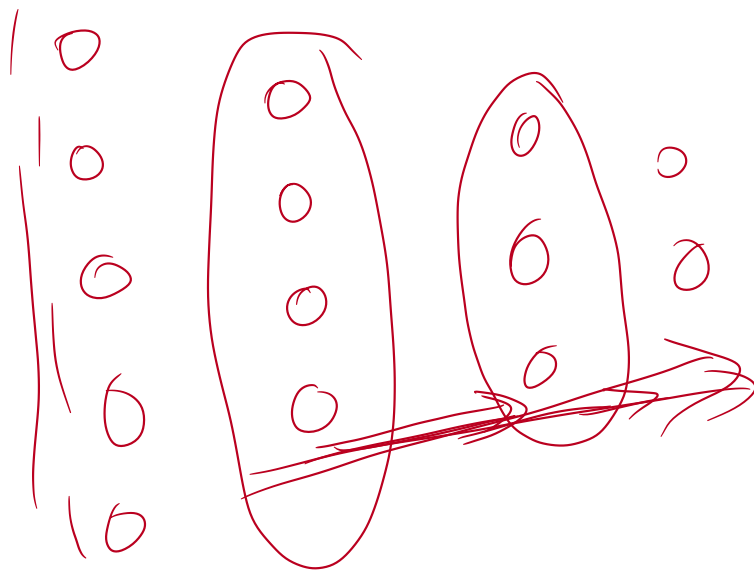
y_1
 y_2
 $\vdots$
 y_{400}

L_1
 L_2

~~400 x 5 = 2000~~

L

~~1 kw~~
~~5~~

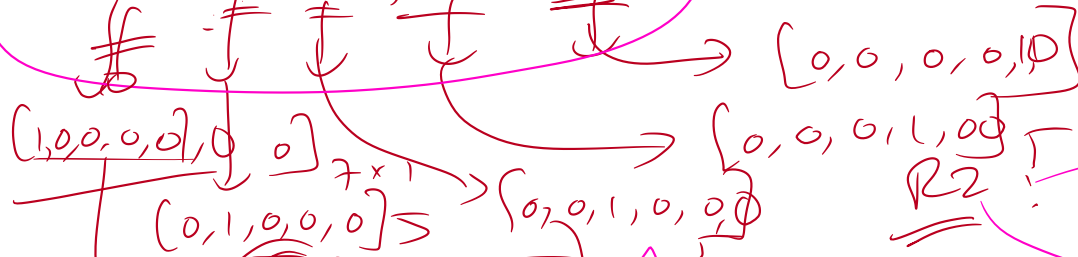


Sentiment Analysis

P1

"It is a good movie"

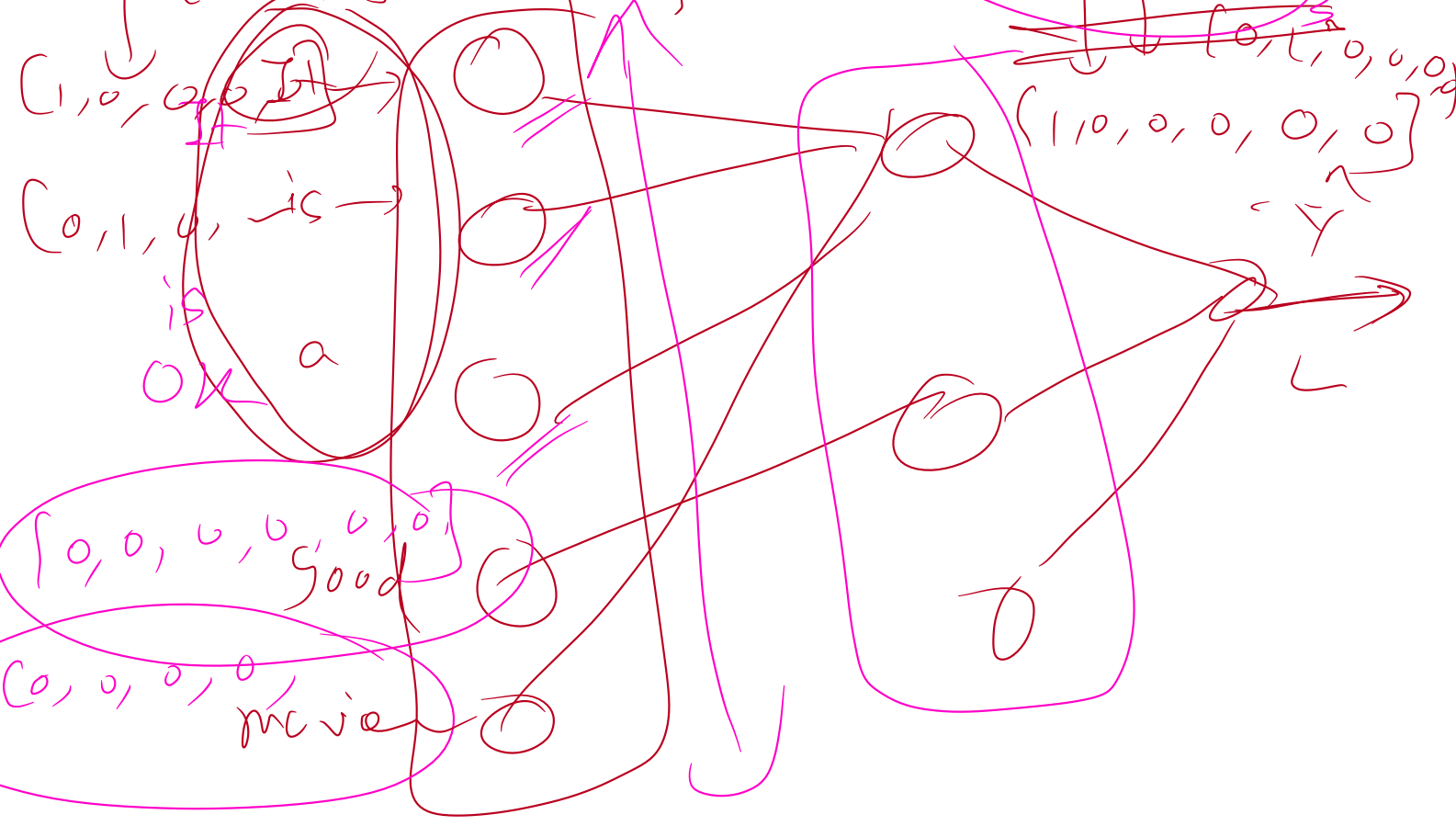
Sen 1



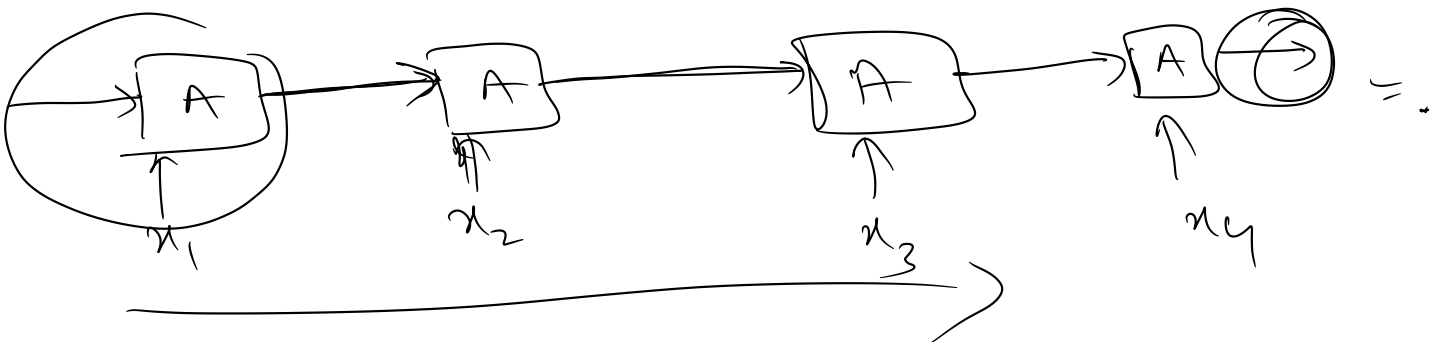
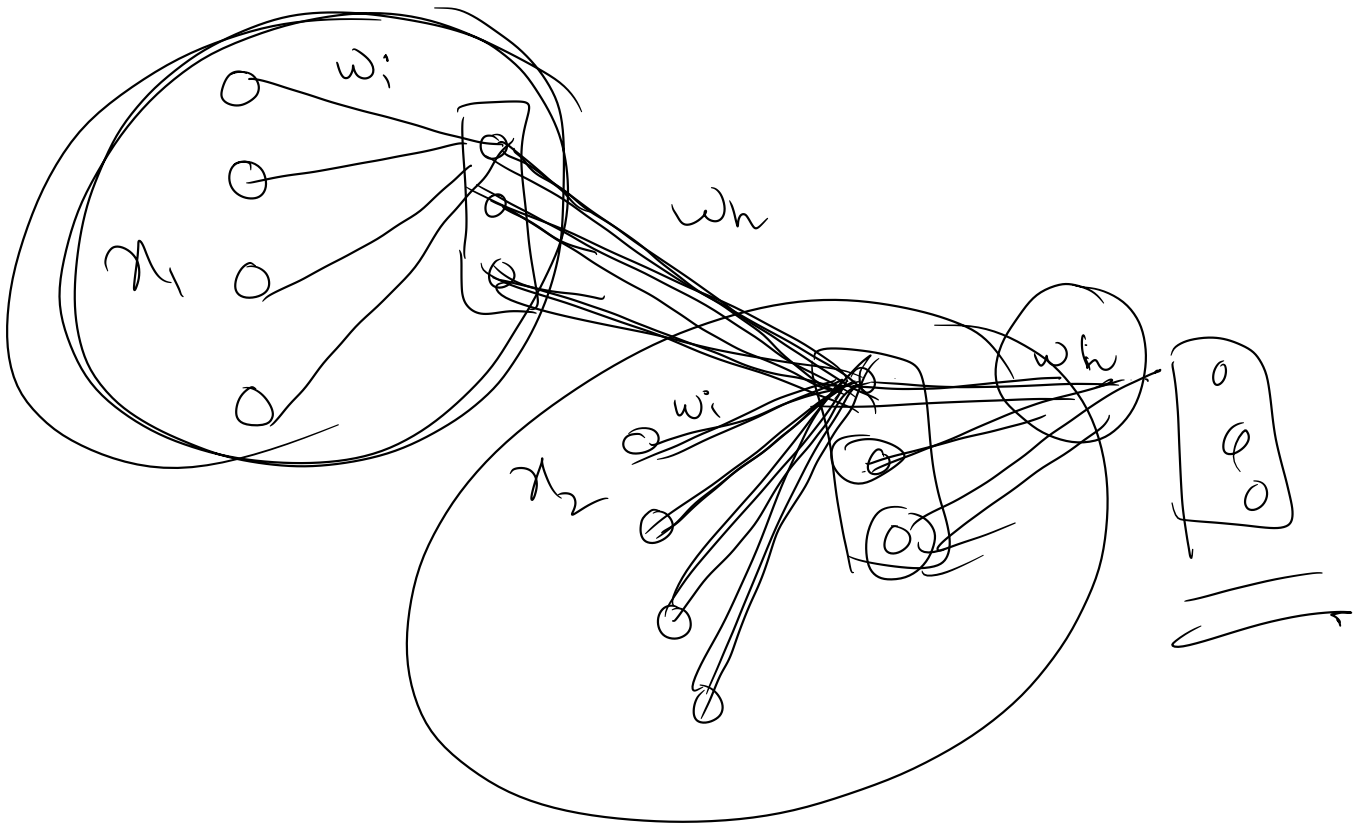
$[0, 0, 0, 0, 0, 1]$

P2

"It is OK"



My name is
— T.S



Sequential Neural Networks

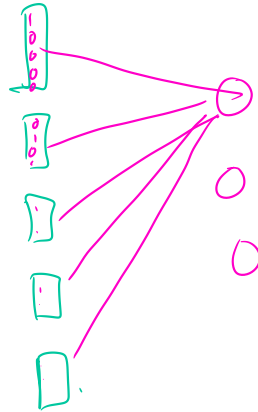
Why to avoid ANNs for sequential data??

For Sentiments Analysis, let's say the dataset has

Sentences like

S_1 : Basudev is studies at IITK } We assign
 S_2 : " " " " } vectors to
 S_3 : " " " " } all words.

Basudev: $[1\ 0\ 0\ 0\ 0]$
 is: $[0\ 1\ 0\ 0\ 0]$
 studies: $[0\ 0\ 1\ 0\ 0]$
 at: $[0\ 0\ 0\ 1\ 0]$
 IITK: $[0\ 0\ 0\ 0\ 1]$



But now let's say, the next input sentence has 10 words.

The input sizes would differ
 So, problem in taking inputs

We might do zero padding.

But it would make it computationally expensive

Why zero padding becomes an issue?? & sparsity in data increases

→ Dense layers expect dense inputs.

→ Sparsity of data ↑: reduces effective information per sample.

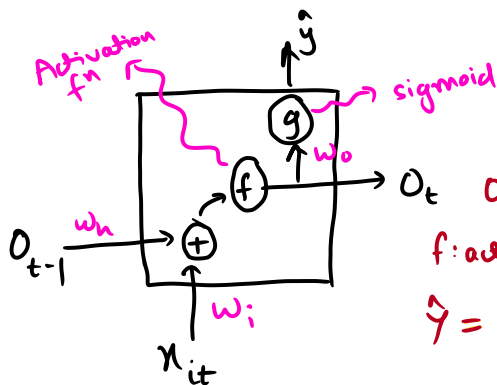
→ Zeros are treated as real data, not padding

→ Many weights never receive useful gradients.

(The corresponding weights barely get updated).

Recurrent Neural Networks

Architecture:



$$O_t = f(O_{t-1} w_h + x_{it} w_i)$$

f : activation fn like tanh.

$$\hat{y} = g(O_t w_o)$$

↳ Sigmoid fn.

After all time steps are complete.

$$L = -y_i \log \hat{y}_i - (1 - y_i) \log (1 - \hat{y}_i)$$

"Recurrent" term is used because the hidden layers appear multiple times.

The RNN "unfolds" through time.

"Back propagation through time" happens here: the algo is "unrolled" across time steps, allowing gradients (errors) to flow backward not just through layers but also through the sequence's history.

Why can't RNNs do machine translation?

An RNN can't produce an output, if we don't feed it an input at every time step.

Let's say we are doing English to Hindi.
Both the languages have different sentence structures.

So, we can't start translating right away at every time step.

We have to capture the meaning first.

So, we have to force our RNN to generate <PAD> or some dummy token till we feed the entire sentence into it.

So, the model doesn't have actual values to BPTT.

To force our RNN;

After final <EOS> of English should output <SOS> of French.

The model has to switch from Eng to French and we have to give the French sentence as input at every time step word by word.

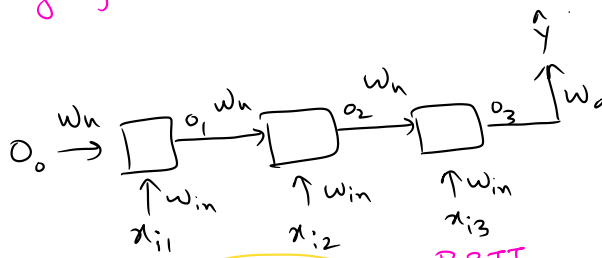
Major Issues with RNNs

→ Problem of long term dependencies.

→ Stagnated training.

Long term dependencies arise due to vanishing gradient problem.

exploding gradients.



$$\frac{\partial L}{\partial w_{in}} = \underbrace{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_{in}}}_{\text{Short term dependency term}} + \underbrace{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_{in}} + \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_{in}}}_{\text{long term dependency}}$$

If we use activation f^n like

tanh or sigmoid

vanishing gradient

Solⁿ: ReLU or Leaky ReLU
Better weight init.

If activation f^n like

ReLU or Leaky ReLU

exploding gradient

Solⁿ: Gradient Clipping (we set the max^m gradient)
Controlled Learning Rate.

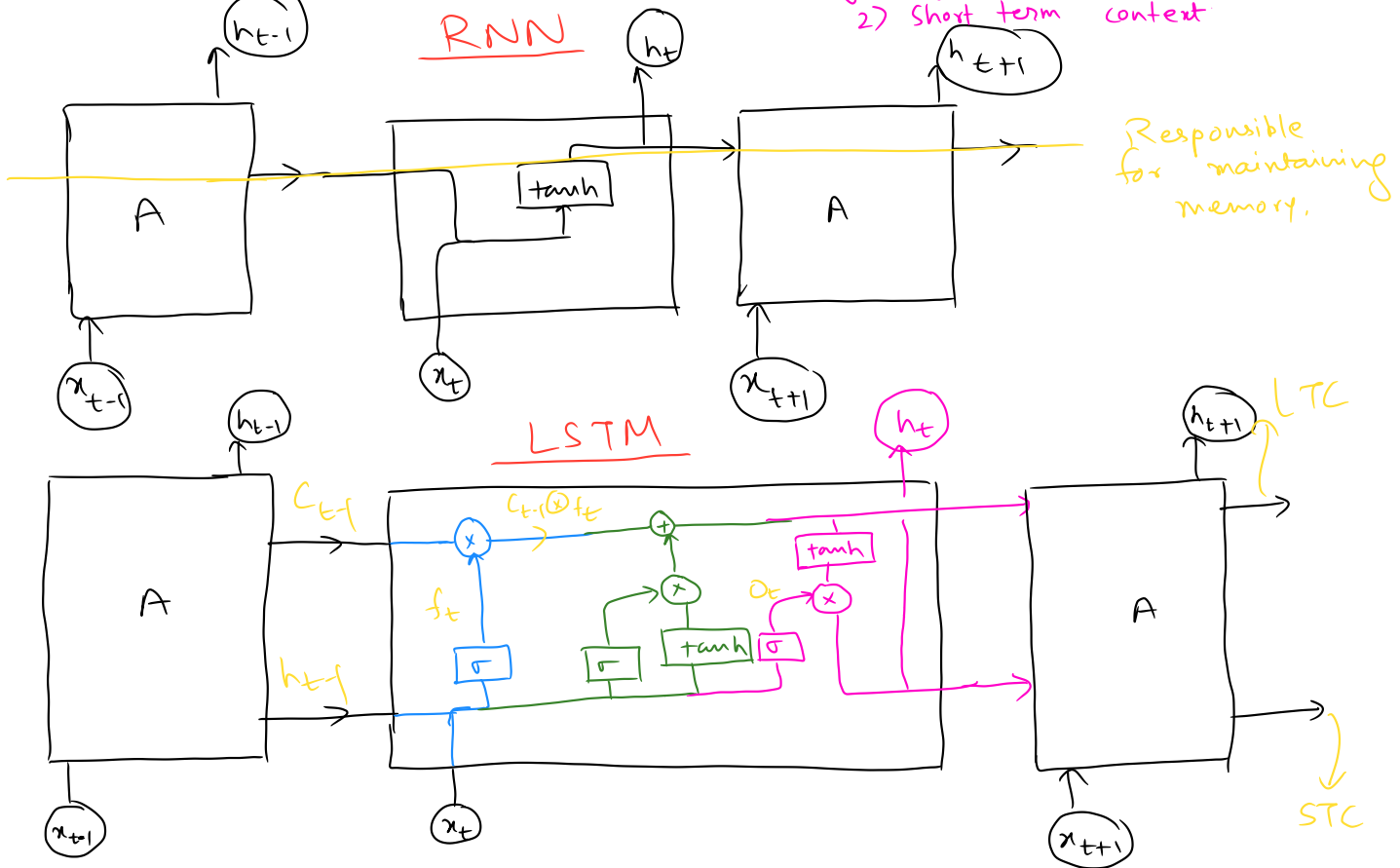
weights may remain near their initialization

chaotic gradient descent, weights become very large

Long Short-Term Memory (LSTM)

RNN maintains memory only by the hidden state line.

It has the pressure of maintaining 1) long term context
2) short term context



Forget gate
what to remove
from LTM based
on STC & current
input.

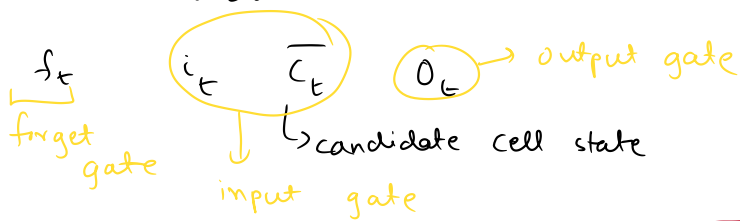
input gate
what to add to
LTM

output gate
Based on LTM
& current
input gives
output
also creates
STM

LTC : C_{t-1} Cell State
STC : h_{t-1} hidden State
Input : x_t

Total inputs

given C_t & h_t
updated created



$\otimes$: point wise multiplication

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \otimes \begin{bmatrix} 3 & 4 & 5 \end{bmatrix} = \begin{bmatrix} 3 & 8 & 15 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \otimes \begin{bmatrix} 3 & 4 & 5 \end{bmatrix} = \begin{bmatrix} 4 & 6 & 8 \end{bmatrix}$$

$[\tanh(1), \tanh(2), \tanh(3)]$

tanh : Neural N with tanh activation f^n

} Number of nodes same throughout lstm

Number of hyperparameter nodes

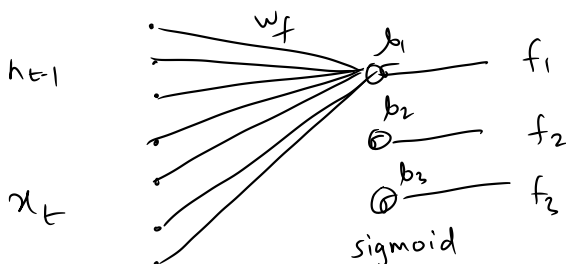
Number of nodes in hidden layers = dimensions of $\ell_{t-1}, h_{t-1}, f_{t-1}, \bar{c}_t, \dot{e}_t, o_t$

Decided by x_t & h_{t-1}

let's No. of nodes' dim = 3

say $\dim(h_{t-1}) = 3$ ←

$$f_t = \sigma \left(\underset{3 \times 7}{w_f} \underset{\substack{7 \times 1 \\ \text{---} \\ 3 \times 1}}{[h_{t-1}, x_t]} + \underset{3 \times 1}{b_f} \right)$$



$$f_t = [f_1 \quad f_2 \quad f_3]$$

↳ forget gate.

Input Gate

- 1) Cal^c candidate cell state $\tilde{C}_t$
- 2) Cal^c i_t decides which info of $\tilde{C}_t$ should be added to C_t
- 3) C_t

$$\tilde{C}_t = \tanh(W_C [h_{t-1}, x_t] + b_C)$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t \otimes i_t = \tilde{C}_t^* \quad (\text{filtered candidate state})$$

This solves Vanishing Gradient Problem

If LSTM feeds the initial information is imp;

then it would set

$$f_t = [1, 1] \quad \text{Example dim} = 3$$

$$i_t = [0, 0, 0] - \text{Example}$$

$$\text{So, } C_{t-1} = C_t$$

$$C_t = f_t \otimes C_{t-1} \oplus i_t \otimes \tilde{C}_t \rightarrow \text{Cell State}$$

Output Gate

$$h_t = O_t \otimes \tanh(C_t)$$

$\tanh$ keeps a safe range of $[-1, 1]$

- $\tanh$ is used because it keeps the hidden activations bounded
- stabilizes training by preventing exploding gradients
- allows gates to control information flow
- exposes only a safe, filtered version of ^{internal} memory

Gated Recurrent Units

LSTM: Complex architecture, more number of parameters.
↳ 3 gates (Training Time more)

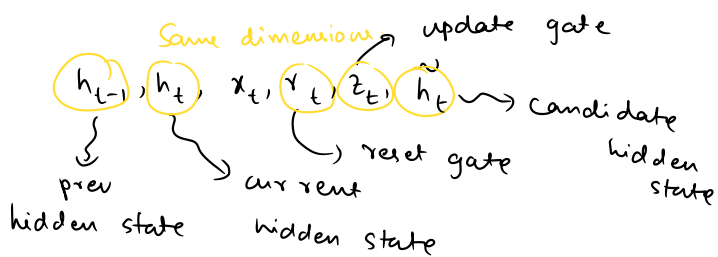
GRU: simpler architecture, lesser parameters.
2 gates So, reduces training time

LSTM: STC & LTC is maintained separately

GRU: Only one hidden state

↳ Reset Gate & Update Gate

We give h_{t-1} & x_t as input.

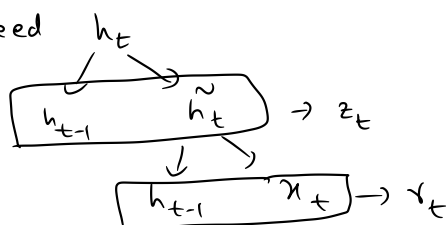


σ , $\tanh$ have same number of nodes = dim ($h_t, x_t, \dots$)

take We text, we use text vectorization techniques

- Steps
- 1) calc. r_t
 - 2) $\tilde{h}_t$
 - 3) z_t
 - 4) h_t

We need



from prev. hidden state (h_{t-1}) & current input (x_t): we form candidate ($\tilde{h}_t$) memory

$\tilde{h}_t$ is inclined towards to current input

h_{t-1} $\tilde{h}_t$ (Balanced) $\rightarrow h_t$

$h_{t-1} \xrightarrow{r_t} \tilde{h}_t \xrightarrow{z_t} h_t$

Overall flowchart

$$r_t = \sigma(w_r [h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(w_c [h_{t-1} \otimes r_t, x_t] + b_c)$$

$$h_t = (1 - z_t) \otimes h_{t-1} \oplus z_t \otimes \tilde{h}_t$$

z_t is high it favours candidate hidden state

z_t is low: favours prev hidden state

Encoder - Decoder

For NLP tasks like machine translation

where input & output have varying lengths.

Let's say English to Hindi

English: I want to really back home 7 tokens
 Hindi: मुझे घर जाना है <start> <end> 6 tokens

OHE for vectorization

English: I want to really back home 7 tokens

Hindi: मुझे घर जाना है <start> <end> 6 tokens

(We calc error across all time steps for BProp)

